

SQL

Complete Syllabus & Study Guide



BROWSEJOBS

Module 1: Introduction to Databases & SQL

Fundamentals

What is SQL?

SQL (Structured Query Language) is the standard language for managing and querying data stored in relational database management systems (RDBMS). Data is organized in **tables** consisting of rows (records) and columns (fields). SQL follows a relational model where multiple tables are connected through relationships, enabling powerful data retrieval and manipulation across complex datasets.

SQL has been the backbone of data management since the 1970s, and despite the rise of NoSQL systems, it remains the most widely used data language in the world. Mastering SQL opens doors to roles in data analysis, backend development, data engineering, and business intelligence.

SQL vs. NoSQL — Know the Difference

Choosing the right database type depends on your use case. SQL databases excel at structured, transactional workloads while NoSQL systems handle scale and flexibility. Here is a direct comparison of both paradigms.

Feature	SQL (Relational)	NoSQL (Non-Relational)
Data Storage	Tables with rows and columns	Key-value pairs, documents, graphs
Schema	Fixed and predefined	Dynamic / flexible (schema evolution)
Scalability	Vertical (add more RAM, CPU)	Horizontal (distributed architecture)
Query Language	SQL	Varies (e.g., BSON for MongoDB)
Examples	MySQL, PostgreSQL, SQL Server, Oracle	MongoDB, Cassandra, DynamoDB
Best For	Transactions (banks, e-commerce)	Big data, analytics, AI/ML

  **Real-World Scenario:** Amazon uses SQL databases (like Aurora/PostgreSQL) for transactional operations — processing orders, payments, and inventory updates where data consistency is critical. Simultaneously, they use NoSQL (like DynamoDB) for their recommendation engine and session management where flexible schema and horizontal scaling are needed.



CRUD Operations & Types of Tables

The Four CRUD Operations

Every database application revolves around these four fundamental operations. They map directly to SQL commands used throughout this course.

CREATE

INSERT — Add new records

READ

SELECT — Query data

UPDATE

UPDATE — Modify records

DELETE

DELETE — Remove records

Types of Tables

Fact Tables

Contain quantitative/measurable data such as sales amounts, revenue, and quantities sold. The core tables in analytical databases.

Factless Fact Tables

Track events or associations without measurable values — for example, which student attended which class on which date.

Dimension Tables

Contain descriptive data that provides context — customer names, product details, geographic information, and dates.

Module 2: Database Design — Keys, Constraints & Relationships

A well-designed database begins with understanding keys and constraints. Keys define how rows are uniquely identified and how tables relate to one another. Constraints enforce business rules at the database level, ensuring data quality before data even reaches your application layer.

Key Type	Description	Rules
Primary Key	Uniquely identifies each row	Unique; cannot be NULL; one per table
Foreign Key	References a primary key in another table	Multiple allowed; can be NULL
Unique Key	Ensures all values in a column are distinct	Multiple allowed; can have NULL
Surrogate Key	System-generated value (auto-increment, UUID)	Used when no natural unique identifier exists
Natural Key	Business-provided identifier	Meaningful to the business domain
Candidate Key	Any column(s) that could serve as primary key	Primary key is chosen from these
Composite Key	Combination of two or more columns	e.g., (order_id + product_id) together unique



Constraints & NULL Handling

Constraints are rules enforced at the database level to maintain data integrity. They act as a safety net — preventing bad data from entering your tables regardless of how the application behaves. Understanding each constraint and when to apply it is a foundational database design skill.



PRIMARY KEY

Enforces uniqueness and NOT NULL simultaneously. Every table should have one.

Example: `employee_id INT PRIMARY KEY`



FOREIGN KEY

Maintains referential integrity between tables.

Ensures every child record points to a valid parent record.



UNIQUE

Prevents duplicate values in a column. Multiple unique constraints allowed per table. Example:

`phone VARCHAR(20) UNIQUE`



NOT NULL

Ensures a column always has a value. Example:

`name VARCHAR(100) NOT NULL`



CHECK

Validates data against a custom condition.

Example: `CHECK (salary > 0)`



DEFAULT

Sets a fallback value when none is provided.

Example: `status VARCHAR(20) DEFAULT 'active'`



NULL Handling Tip: SQL Server treats `NULL = NULL` as `TRUE`, allowing only **ONE** NULL in a **UNIQUE** column. PostgreSQL and MySQL treat `NULL <> NULL`, allowing **MULTIPLE** NULLs in a **UNIQUE** column. Know your database engine!

Module 3: Data Definition Language (DDL) — Creating Tables

DDL commands define the structure of your database. The `CREATE TABLE` statement specifies the table name, columns, data types, and constraints. Getting your table structure right from the start saves enormous effort in later migrations and refactoring.

```
CREATE TABLE Employees (  
  employee_id INT PRIMARY KEY,  
  first_name VARCHAR(255),  
  last_name VARCHAR(255),  
  email VARCHAR(255),  
  phone_number VARCHAR(20),  
  hire_date DATE,  
  job_id VARCHAR(10),  
  salary DECIMAL(10, 2),  
  department_id INT  
);
```

Data Type	Description	Example Values
INT	Whole numbers	1, 42, -100
DECIMAL(p,s)	Precise decimal numbers	75000.00, 3.14
VARCHAR(n)	Variable-length text up to n characters	'John', 'New York'
DATE	Calendar date	'2024-01-15'
TIMESTAMP	Date and time combined	'2025-09-01 09:00:00'
BOOLEAN	True/False values	TRUE, FALSE



Creating Related Tables with Foreign Keys

In a relational database, tables reference each other using foreign keys. This example shows how an Employees table relates to a Departments table, and how a Job_History table uses a composite primary key to track career changes over time.

```
CREATE TABLE Departments (  
  department_id INT PRIMARY KEY,  
  department_name VARCHAR(100),  
  location VARCHAR(100),  
  manager_id INT  
);  
  
-- Job_History: composite primary key  
CREATE TABLE Job_History (  
  employee_id INT,  
  start_date DATE,  
  end_date DATE,  
  job_id VARCHAR(10),  
  department_id INT,  
  PRIMARY KEY (employee_id, start_date),  
  FOREIGN KEY (employee_id)  
    REFERENCES Employees(employee_id),  
  FOREIGN KEY (department_id)  
    REFERENCES  
  Departments(department_id)  
);
```

Why Composite Keys?

The Job_History table uses a composite primary key of (employee_id, start_date) because an employee can have multiple job records — but only one starting on any given date. This combination is always unique.



Real-World Scenario: An HR system needs tables for Employees, Departments, Jobs, and Job_History. Foreign keys ensure every job history record maps to a valid employee, department, and job — preventing orphaned records.



Module 4: Data Manipulation — SELECT, WHERE, ORDER BY, LIMIT

The **SELECT** statement is the most frequently used SQL command. Mastering it — along with **WHERE**, **ORDER BY**, and **LIMIT** — gives you the ability to retrieve exactly the data you need from any table.

1

SELECT

Retrieves data from one or more tables. Use specific column names in production — avoid **SELECT *** for better performance.

```
SELECT first_name, last_name, salary
FROM employees;
```

2

WHERE

Filters rows based on conditions. Only rows satisfying the condition are returned.

```
SELECT * FROM employees
WHERE salary > 75000;
```

3

ORDER BY

Sorts results in ascending (ASC, default) or descending (DESC) order.

```
SELECT first_name, salary
FROM employees
ORDER BY salary DESC;
```

4

LIMIT & OFFSET

Restricts row count — essential for pagination and finding top/bottom results.

```
SELECT first_name, salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1; -- 2nd highest
```



Module 5: Operators & Filtering — AND, OR, NOT, LIKE, IN, BETWEEN

SQL provides a rich set of filtering operators that let you express complex conditions with clean, readable syntax. These operators are the building blocks of every WHERE clause you will ever write.

Logical Operators

```
-- AND: both conditions true
SELECT * FROM employees
WHERE salary > 75000
  AND first_name = 'Michael';
```

```
-- OR: at least one condition true
SELECT * FROM employees
WHERE salary > 75000
  OR first_name = 'Michael';
```

```
-- NOT: negates condition
SELECT * FROM employees
WHERE NOT salary > 75000;
```

IN Operator

```
SELECT * FROM employees
WHERE last_name IN ('Doe','Smith','Davies');
```

BETWEEN Operator

```
SELECT * FROM employees
WHERE department_id BETWEEN 10 AND 40;
```

LIKE Wildcards

Wild card	Meaning	Example
%	Zero or more characters	LIKE 'M%' → Michael, Mary
_	Exactly one character	LIKE '____' → John, Jane
[]	Any char in set (SQL Server)	LIKE '[aeiou]%'

```
-- Names starting with 'M'
SELECT * FROM employees
WHERE first_name LIKE 'M%';
```

```
-- PostgreSQL pattern
SELECT * FROM employees
WHERE first_name
  SIMILAR TO '(J|K)%';
```



Real-World: CRM search uses LIKE for partial name search, IN for city filter, and BETWEEN for date range filtering — all in a single query.



Module 6: Aggregate Functions — COUNT, SUM, AVG, MIN, MAX

Aggregate functions perform calculations on a set of rows and return a single result. They are the foundation of every analytical query — from monthly financial reports to performance dashboards. These functions always operate on a column's values across multiple rows.

COUNT()

Total Rows

Count all records in a table or group

SUM()

Total Sum

Add all values in a numeric column

AVG()

Average

Calculate mean value across all rows

MIN()

Minimum

Find the smallest value in a column

MAX()

Maximum

Find the largest value in a column

```
SELECT COUNT(*) AS headcount,  
       SUM(salary) AS total_payroll,  
       AVG(salary) AS avg_compensation,  
       MIN(salary) AS lowest_salary,  
       MAX(hire_date) AS newest_hire  
FROM employees;
```



Real-World Scenario: A CFO needs a monthly snapshot — total payroll (SUM), headcount (COUNT), average compensation (AVG), highest salary (MAX) for budget planning, and newest hire date (MAX on hire_date) to track recruitment velocity. These aggregates form the backbone of every financial dashboard.



Module 7: GROUP BY & HAVING

GROUP BY and **HAVING** are the pair that unlocks true analytical power in SQL. Together they let you summarize data by category and then filter on those summaries — a pattern used in virtually every business intelligence query ever written.

GROUP BY — Aggregating by Category

GROUP BY groups rows sharing a common value and applies aggregate functions to each group separately.

```
SELECT d.department_name,  
       AVG(e.salary)    AS avg_salary,  
       COUNT(e.employee_id) AS headcount  
FROM departments d  
INNER JOIN employees e  
  ON d.department_id = e.department_id  
GROUP BY d.department_name;
```

HAVING — Filtering Groups

HAVING filters groups **AFTER** aggregation. **WHERE** filters individual rows **BEFORE** grouping. This is a critical distinction.

-- Departments with total salary > 80K

```
SELECT department_id,  
       SUM(salary) AS total_salary  
FROM employees  
GROUP BY department_id  
HAVING SUM(salary) > 80000;
```

-- WRONG: Cannot use aggregate in
WHERE

-- WHERE SUM(salary) > 80000 → ERROR



Real-World Scenario: A regional sales director wants to identify underperforming territories. They **GROUP BY** region, calculate **SUM(revenue)** and **COUNT(deals)**, then use **HAVING SUM(revenue) < 100000** to find regions falling below target. This two-step process is impossible with **WHERE** alone.

Module 8: Joins — INNER, LEFT, RIGHT, FULL OUTER, SELF & CROSS

Joins are the most powerful feature of relational databases. They allow you to combine data from multiple tables based on related columns — turning normalized, separated data back into meaningful, unified result sets. Understanding every join type is non-negotiable for any SQL practitioner.

Join Type	Returns	NULL Handling
INNER JOIN	Only matching rows from both tables	Excludes non-matching rows entirely
LEFT JOIN	All rows from left + matches from right	NULL where right table has no match
RIGHT JOIN	All rows from right + matches from left	NULL where left table has no match
FULL OUTER JOIN	All rows from both tables	NULL on both sides where no match
SELF JOIN	Table joined with itself	Used for hierarchical data
CROSS JOIN	Cartesian product of both tables	Every row paired with every other row



Join Examples in Action

INNER JOIN

```
SELECT e.first_name, e.last_name,  
       d.department_name  
FROM employees e  
JOIN departments d  
  ON e.department_id = d.department_id;
```

LEFT JOIN — All Departments

```
SELECT d.department_name,  
       e.first_name, e.last_name  
FROM departments d  
LEFT JOIN employees e  
  ON d.department_id = e.department_id;  
-- Includes depts with no employees (NULL)
```

SELF JOIN — Manager Hierarchy

```
SELECT e.empname AS employee,  
       m.empname AS manager  
FROM employee e  
JOIN employee m  
  ON e.managerid = m.empid;
```

Multi-Table Join

```
SELECT e.first_name, e.last_name,  
       d.department_name, j.job_title  
FROM employees e  
JOIN departments d  
  ON e.department_id = d.department_id  
JOIN jobs j  
  ON e.job_id = j.job_id;
```

CROSS JOIN — Cartesian Product

```
-- All color-size combinations  
SELECT c.colour_name, s.size_name  
FROM colors c  
CROSS JOIN sizes s;  
-- red-small, red-large, blue-small...
```



IPL Fixtures: A SELF JOIN on teams with `t1.team_id < t2.team_id` generates unique match pairs without duplicates. With 4 teams, that's 6 unique matchups: $n(n-1)/2$.



Module 9: Subqueries & Nested Queries

A subquery is a query nested inside another query. The inner query executes first, and its result is used by the outer query. Subqueries are incredibly versatile — they can appear in the `WHERE` clause, the `FROM` clause, or even the `SELECT` clause itself, enabling multi-layered logic within a single SQL statement.

1

Subquery in WHERE

```
-- Most recently hired employee
SELECT first_name, last_name, hire_date
FROM employees
WHERE hire_date = (
    SELECT MAX(hire_date) FROM
employees
);

-- Above-average earners
SELECT first_name, last_name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary) FROM employees
);
```

2

Subquery in FROM (Derived Table)

```
-- Departments with avg salary > 80K
SELECT dept_stats.department_id,
       dept_stats.avg_salary
FROM (
    SELECT department_id,
           AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department_id
) AS dept_stats
WHERE dept_stats.avg_salary > 80000;
```



Real-World Scenario: An HR team identifies employees eligible for a performance bonus — those earning above the department average. The subquery calculates each department's average salary, and the outer query finds employees whose salary exceeds that average.



Module 10: Common Table Expressions (CTEs)

A CTE (Common Table Expression) is a temporary named result set defined using the `WITH` keyword. It exists only for the duration of the query execution. CTEs dramatically improve the readability and maintainability of complex queries by breaking them into named, logical steps – similar to how functions work in programming languages.

Key Benefits: Improved readability, reusability within the query, ability to reference the result multiple times, and cleaner debugging. Note: A CTE does NOT save data or the query itself – it only lasts during execution.

1

Define the CTE

Use the `WITH` keyword to name your temporary result set and write the inner query

2

Apply Filters

The outer query references the CTE like any regular table, adding further conditions

3

Get Results

Execute and retrieve clean, readable output without nested subquery clutter

```
-- Employees earning >70K whose name starts with A
WITH greaterthan70 AS (
  SELECT employee_id, first_name, last_name, salary
  FROM employees
  WHERE salary > 70000
)
SELECT * FROM greaterthan70
WHERE first_name LIKE 'A%' OR last_name LIKE 'A%'
ORDER BY salary DESC
LIMIT 1;
```



CTEs with Aggregation & Window Functions

CTEs become especially powerful when combined with aggregations and window functions. They allow you to compute intermediate results and then filter or transform them in a second step – a pattern that is otherwise messy with subqueries. This mirrors the ETL (Extract, Transform, Load) logic used in data engineering pipelines.

CTE with Aggregation

```
-- Departments with total salary > 80K
WITH cte AS (
  SELECT department_id,
         SUM(salary) AS total_salary
  FROM employees
  GROUP BY department_id
)
SELECT * FROM cte
WHERE total_salary > 80000;
```

CTE with Window Functions

```
-- Employees above their dept average
WITH cte AS (
  SELECT first_name, last_name,
         salary, department_id,
         AVG(salary) OVER (
           PARTITION BY department_id
         ) AS avg_dept_salary
  FROM employees
)
SELECT * FROM cte
WHERE salary > avg_dept_salary;
```



Real-World Scenario: In a data engineering pipeline, CTEs act like intermediate transformation steps. CTE1 cleans raw transaction data, CTE2 joins it with customer data, CTE3 aggregates by region, and the final SELECT filters for high-value regions. This mirrors ETL logic and makes complex transformations readable.

Module 11: Window Functions — RANK, DENSE_RANK, ROW_NUMBER, LAG, LEAD

Window functions perform calculations across a set of rows related to the current row, without collapsing them into a single output row. Unlike `GROUP BY`, window functions retain every row in the result — making them ideal for rankings, running totals, and period-over-period comparisons. They are one of the most powerful and frequently tested SQL concepts in technical interviews.

Function	Behavior with Ties	Example Result (50K, 49K, 45K, 45K, 20K)
<code>RANK()</code>	Skips ranks after ties	1, 2, 3, 3, 5
<code>DENSE_RANK()</code>	No gaps after ties	1, 2, 3, 3, 4
<code>ROW_NUMBER()</code>	Unique number for every row	1, 2, 3, 4, 5

```
-- Find the 2nd highest salary
WITH cte AS (
  SELECT employee_id, first_name, last_name, salary,
  DENSE_RANK() OVER (ORDER BY salary DESC) AS ranking
  FROM employees
)
SELECT * FROM cte WHERE ranking = 2;
```

```
-- 2nd highest salary BY DEPARTMENT
WITH cte AS (
  SELECT department_id, employee_id, first_name, salary,
  DENSE_RANK() OVER (PARTITION BY department_id
  ORDER BY salary DESC) AS ranking
  FROM employees
)
SELECT * FROM cte WHERE ranking = 2;
```



LAG & LEAD — Time-Series Analysis

`LAG()` accesses data from a previous row within a partition. `LEAD()` accesses data from the next row. These functions are indispensable for comparing current values to past or future values — essential for churn analysis, sales trends, and behavioral analytics.

LAG — Previous Row Access

```
-- Customer's previous transaction
SELECT customer_id, sale_date, amount,
       LAG(amount) OVER (
         PARTITION BY customer_id
         ORDER BY sale_date
       ) AS prev_amount
FROM sales;

-- Days between transactions
SELECT customer_id, sale_date,
       DATEDIFF(day,
        LAG(sale_date) OVER (
          PARTITION BY customer_id
          ORDER BY sale_date),
        sale_date) AS days_between
FROM sales;
```

LEAD — Next Row Access

```
-- Salary increment percentage
WITH cte AS (
  SELECT employee_id, first_name,
         salary,
         LEAD(salary) OVER (
           PARTITION BY employee_id
           ORDER BY hire_date
         ) AS next_salary
  FROM employees
)
SELECT employee_id, salary,
       next_salary,
       ROUND(((next_salary - salary)
        / salary) * 100, 2)
       AS percent_increase
FROM cte
WHERE next_salary IS NOT NULL;
```



Real-World Scenario: An e-commerce platform uses `LAG()` to calculate days between each customer's purchases. Gaps exceeding 90 days flag the customer as "at-risk." `LEAD()` predicts the next expected purchase date. These window functions power retention dashboards and automated re-engagement email campaigns.

Window Functions vs. Aggregate Functions

Both window functions and aggregate functions perform calculations across a set of rows, but their core distinction lies in how they present their results. Understanding this fundamental difference is crucial for effective data analysis and crafting efficient SQL queries. While aggregate functions summarize data and reduce the number of rows, window functions enrich each individual row with contextual calculations.

Aggregate Functions (e.g., `COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`) operate on a group of rows defined by the `GROUP BY` clause. They condense these groups into a single summary row, meaning the output will typically have fewer rows than the input. For instance, you might use an aggregate function to find the total sales for each product category, resulting in one row per category.

Window Functions (e.g., `RANK()`, `ROW_NUMBER()`, `LAG()`, `LEAD()`) also perform calculations on a set of rows, known as a "window," but crucially, they do not collapse the rows. Instead, they add a new calculated column to each row of the original result set. This allows for complex calculations like running totals, moving averages, or comparing a row's value to a previous or subsequent row within a defined partition, all while retaining the granular detail of the original data.

Module 12: CASE Statements & Conditional Logic

The `CASE` statement adds if-then-else logic directly within SQL queries, allowing you to categorize, label, or transform data conditionally. It eliminates the need to pull raw data into an application just to apply business rules — the transformation happens at the database layer, where it belongs. `CASE` statements work inside `SELECT`, `WHERE`, `ORDER BY`, and even aggregate functions.

Salary Band Categorization

```
SELECT first_name, salary,
CASE
  WHEN salary >= 85000 THEN 'High'
  WHEN salary BETWEEN 63000
    AND 84999 THEN 'Medium'
  ELSE 'Low'
END AS salary_category
FROM employees;
```

Role-Based Bonus Calculation

```
SELECT first_name, job_id, salary,
CASE
  WHEN job_id = 'DEV01'
    THEN salary * 0.10
  WHEN job_id = 'HR01'
    THEN salary * 0.15
  ELSE 0
END AS bonus
FROM employees;
```



Module 13: Handling Duplicates

Duplicate records are a persistent data quality problem — they appear during data ingestion, system migrations, and API retries. SQL provides two primary approaches to finding and removing duplicates: `GROUP BY + HAVING` for quick detection, and `ROW_NUMBER()` for flexible, targeted removal while preserving the desired record.

Method 1: GROUP BY + HAVING

Quick way to identify duplicated values in a column. Best for simple, single-column duplicate detection.

```
SELECT email,
COUNT(*)
FROM employees
GROUP BY email
HAVING COUNT(*) > 1;
```

Method 2: ROW_NUMBER (More Flexible)

Assigns a sequential number within each group. Rows with `row_num > 1` are duplicates. Allows multi-column partitioning.

```
WITH cte AS (
  SELECT employee_id,
         first_name,
         last_name, email,
         ROW_NUMBER()
         OVER (
           PARTITION BY
             first_name,
             last_name,
             email
           ORDER BY
             employee_id
         ) AS row_num
  FROM employees
)
SELECT * FROM cte
WHERE row_num > 1;
```

Removing Duplicates (Keep First)

Extend the CTE approach to DELETE duplicate rows while keeping the earliest record.

```
WITH cte AS (
  SELECT employee_id,
         ROW_NUMBER()
         OVER (
           PARTITION BY
             first_name,
             last_name, email
           ORDER BY
             employee_id
         ) AS row_num
  FROM employees
)
DELETE FROM
employees
WHERE employee_id
IN (
  SELECT employee_id
  FROM cte WHERE
  row_num > 1
);
```



Module 14: Stored Procedures

Stored procedures are precompiled groups of SQL statements stored in the database as a single named unit. They offer faster execution (precompiled query plan), enhanced security against SQL injection (parameters are sanitized automatically), and reusability — write once, call anywhere. They are especially valuable for complex multi-step operations like payroll processing or data migrations.

SQL Server Syntax

```
-- Create: Update employee salary
CREATE PROCEDURE updateemployeesalary
    @employee_id INT,
    @newsalary DECIMAL(10,2)
AS BEGIN
    UPDATE employees
    SET salary = @newsalary
    WHERE employee_id = @employee_id;
END;

-- Execute it
EXEC updateemployeesalary
    @employee_id = 105,
    @newsalary = 70000.00;
```

MySQL Syntax

```
-- Show employees by department
CREATE PROCEDURE
    showemployeesbydepartment(
        IN dept_name VARCHAR(255))
BEGIN
    SELECT * FROM employees
    WHERE department = dept_name;
END;

CALL showemployeesbydepartment('HR');

-- Insert new employee
CREATE PROCEDURE addemployee(
    IN empid INT, IN empname
    VARCHAR(255),
    IN dept VARCHAR(255), IN salary INT)
BEGIN
    INSERT INTO employees
        (id, name, department, salary)
    VALUES (empid, empname, dept, salary);
END;
```



Real-World Scenario: A payroll system uses stored procedures to process monthly salaries — validating employee status, calculating deductions (tax, insurance), applying bonuses, and inserting records into payroll_history — all in a single atomic transaction. This prevents partial updates and ensures data integrity.

Module 15: Views & Materialized Views

Views and materialized views are two related but fundamentally different tools for simplifying complex queries and improving database architecture. A view is a virtual table — a saved query definition. A materialized view is a physical snapshot — a saved query AND its results stored on disk. Choosing between them depends on how frequently the data changes versus how fast you need reads to be.

Regular Views (Virtual)

Stores only the query definition. Executes the underlying SQL every time you query it. Always shows current data but can be slow for complex queries.

```
CREATE VIEW indiancustomers AS
SELECT * FROM customers
WHERE country = 'India';

-- Query like a table
SELECT * FROM indiancustomers
WHERE age > 18;
```

Materialized Views (Physical)

Stores query AND results on disk. Blazing fast reads — but requires periodic refresh to stay current. Ideal for dashboards and heavy aggregations.

```
CREATE MATERIALIZED VIEW Imployees
AS
SELECT e.first_name, e.last_name,
       d.department_name
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id
WHERE d.department_name = 'IT';

-- Refresh manually
REFRESH MATERIALIZED VIEW
Imployees;
```

Strategy	How It Works	Use Case
Manual Refresh	REFRESH MATERIALIZED VIEW view_name;	On-demand, infrequent updates
Scheduled (Cron)	Cron job: 0 0 * * * (daily at midnight)	Regular intervals (daily/hourly)
Fast/Incremental	Only refreshes changed data using logs	High-frequency, large datasets

Module 16: Indexing & Query Optimization

An index is a data structure (typically B-Tree) that speeds up data retrieval on frequently queried columns — similar to a book's index that lets you find topics without reading every page. Without indexes, the database performs a full table scan on every query, reading every row. With the right indexes, queries that took seconds run in milliseconds.

Clustered vs Non-Clustered

Feature	Clustered	Non-Clustered
Count per table	Only ONE	Multiple allowed
Physical order	Reorders disk data	Separate structure
Leaf nodes	Actual data rows	Pointers to rows
Created by	Auto on PRIMARY KEY	Manually created

Creating Indexes

```
-- Single column index
CREATE INDEX idx_salary
ON employees(salary);

-- Uses index:
SELECT * FROM employees
WHERE salary > 50000;

-- Composite index
CREATE INDEX idx_name
ON employees(first_name, last_name);

-- Uses composite index:
SELECT * FROM employees
WHERE first_name = 'John'
AND last_name = 'Doe';
```

- 1

Use specific column names, not SELECT *

Reduces data transfer and improves performance significantly in production environments.
- 2

Analyze execution plans

Use EXPLAIN ANALYZE (PostgreSQL) to identify full table scans and determine where indexes help most.
- 3

Avoid over-indexing

Too many indexes slow down INSERT/UPDATE/DELETE operations. Index strategically, not exhaustively.



Module 17: Partitioning & Sharding

Partitioning divides large tables into smaller, manageable pieces while keeping them logically as one table. This dramatically improves query performance on large datasets because the database only scans relevant partitions — a technique called "partition pruning." Sharding takes this further by distributing partitions across multiple physical servers.

Range Partitioning by Year

```
CREATE TABLE orders (  
  orderid INT,  
  customerid INT,  
  orderdate DATE  
) PARTITION BY RANGE (YEAR(orderdate)) (  
  PARTITION p2022  
    VALUES LESS THAN (2023),  
  PARTITION p2023  
    VALUES LESS THAN (2024),  
  PARTITION pmax  
    VALUES LESS THAN MAXVALUE  
);
```

```
-- Only scans the 2023 partition  
SELECT * FROM orders  
WHERE YEAR(orderdate) = 2023;
```

Partitioning Strategies

Range

Partition by value ranges — dates, IDs, regions

Hash

Distribute evenly:
`customer_id % 10`
across 10 partitions

List

Partition by discrete values — country, status codes



Real-World Scenario: A telecom company stores billions of call records. They partition by month so queries like "total minutes for January 2025" only scan one partition. Historical partitions older than 2 years are archived to cold storage.

Modules 18 & 19: Normalization & Schema Design

Normalization organizes data by breaking large tables into smaller, related ones to reduce redundancy and improve integrity. Schema design then determines how those normalized (or denormalized) tables are arranged for analytical workloads.

1

1NF — First Normal Form
Every cell must contain atomic (single) values. No multi-valued fields or repeating groups in a column.

2

2NF — Second Normal Form
Satisfies 1NF AND all non-key columns must be fully dependent on the entire primary key. Eliminates partial dependencies.

3

3NF — Third Normal Form
Satisfies 2NF AND removes transitive dependencies. No non-key column should depend on another non-key column. Result: fully normalized tables.

Feature	Star Schema	Snowflake Schema
Structure	Central fact + dimension tables	Fact + normalized sub-tables
Normalization	Denormalized (less normalized)	More normalized
Performance	Faster reads (fewer joins)	Slower reads (more joins)
Best For	OLAP / Analytics / Warehousing	OLTP / Transactional

  **Real-World Scenario:** Amazon Redshift uses star schemas because analytical queries need fast aggregation with minimal joins. The denormalized dimensions trade storage efficiency for query speed — exactly what BI analysts need.



Module 20: OLTP vs. OLAP

OLTP and OLAP represent two fundamentally different database paradigms — one optimized for rapid, transactional writes, the other for complex analytical reads. Modern data architectures use both, with data flowing from OLTP systems into OLAP warehouses through ETL or streaming pipelines.

OLTP

Online Transaction Processing

Day-to-day operations: INSERT, UPDATE, DELETE. Millisecond response. Highly normalized. Examples: MySQL, PostgreSQL, SQL Server. Use case: Banking, e-commerce checkout, order management.

OLAP

Online Analytical Processing

Analytics and reporting: mostly SELECT. Seconds to minutes for complex reads. Less normalized (star schema). Examples: Redshift, Snowflake, BigQuery. Use case: BI dashboards, data science, strategic decisions.



Real-World Scenario — Swiggy: When you place an order on Swiggy, it's processed in their OLTP database (PostgreSQL) — recording the order, updating restaurant availability, deducting from your wallet. Overnight, this data flows into their OLAP warehouse (Redshift) where analysts study average delivery times by city, peak order hours, and customer lifetime value — powering decisions on driver allocation and marketing spend.



Modules 21–22: UNION, UNION ALL & DML Operations

UNION vs. UNION ALL

UNION combines results and removes duplicates. **UNION ALL** keeps ALL rows including duplicates — and is faster because it skips deduplication.

```
-- UNION: unique countries only
SELECT country FROM customers
UNION
```

```
SELECT country FROM suppliers;
```

```
-- UNION ALL: all including duplicates
SELECT country FROM customers
UNION ALL
```

```
SELECT country FROM suppliers;
```

```
-- Combining different data shapes
SELECT cust_id, cust_name,
       NULL AS amount FROM customers
UNION
```

```
SELECT cust_id, NULL AS cust_name,
       amount FROM orders;
```

DROP vs. TRUNCATE vs. DELETE

Command	What It Does	Reversible?
DROP	Deletes entire table structure and data	No (DDL)
TRUNCATE	Removes all rows, keeps structure	No (DDL)
DELETE	Removes specific rows (with WHERE)	Yes (DML)

```
-- DROP: remove table completely
DROP TABLE employees;
```

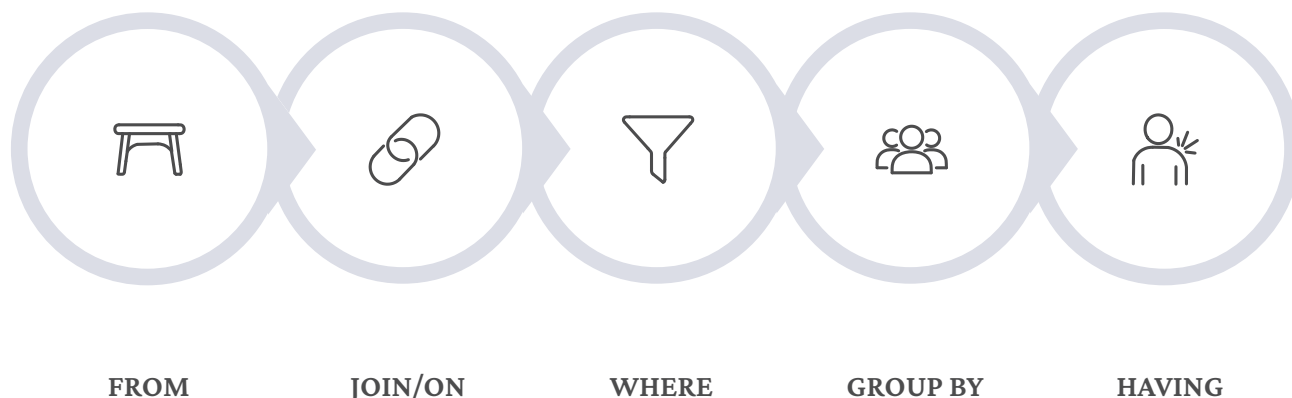
```
-- TRUNCATE: clear data, keep structure
TRUNCATE TABLE employees;
```

```
-- DELETE: remove specific records
DELETE FROM employees
WHERE department_id = 60;
```

 **Tip:** Use UNION ALL when duplicates are acceptable or impossible — it's significantly faster than UNION for large datasets.

Modules 23–24: SQL Execution Order & Data Modelling

Understanding SQL's order of execution is one of the most important concepts for writing correct queries and debugging errors. SQL is NOT executed in the order it is written. Meanwhile, data modelling defines how your entire database system is blueprinted – from high-level concepts to physical implementation.



A critical mnemonic: "**FROM** Japan, **O**nce a **W**ise **G**irl **S**electe**D** **D**istinctly, **O**rdere**D** **L**avishly" – FROM, JOIN, ON, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT. Understanding this order explains why you cannot use a SELECT alias in a WHERE clause – WHERE executes before SELECT sees the alias.

Conceptual Model

High-level entities and relationships. Audience: business stakeholders. No data types – just "what data exists."

Logical Model

Detailed structure with attributes, keys, and relationship types. Audience: data architects. Medium detail – "how data relates."

Physical Model

Actual table names, columns, data types, indexes, and constraints in DDL. Audience: DBAs. Full implementation detail.



Real-World Scenario: When designing a job matching database, the conceptual model identifies entities (Job Seeker, Company, Job Posting, Application). The logical model defines relationships (one company → many postings, one seeker → many applications). The physical model implements this in PostgreSQL with specific data types, indexes on frequently searched fields (skills, location), and partitioning on application_date for performance. This three-level approach ensures alignment between business requirements and technical implementation.